Adversarially Robust Intrusion Detection System

Deep Learning en Cybersécurité — Projet 1

Zakariya El Mansouri

ICCN-INE2

Avril 2026

Plan

1. Contexte et Threat Model
2. Dataset et Prétraitement
3. Modèle Baseline
4. Attaques Adversariales
5. Défenses
6. Comparaison et Résultats
7. Conclusion

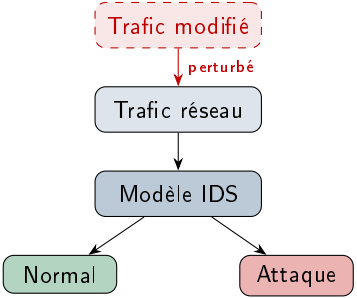
Problématique

Système de Détection d'Intrusion (IDS)

- Surveille le trafic réseau
- Détecte les connexions malveillantes
- Baseé sur l'apprentissage automatique

Problème : exemples adversariaux

Un attaquant peut modifier légèrement son trafic pour tromper le modèle tout en maintenant l'efficacité de l'attaque.



Threat Model

✓ Features manipulables (14/42)

L'attaquant contrôle le timing et le volume :

`dur, spkts, dpkts, sbytes, dbytes, sttl, sload, dload, sinpkt, dinpkt, sjit, djit, smean, dmean`

× Features non manipulables

Changer ces valeurs briserait l'attaque :

`proto, service, state, srcip, dstip, dport`

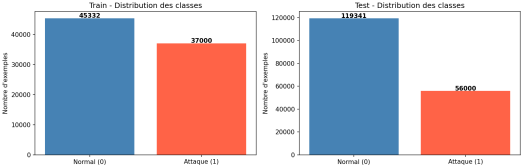
Modèle de l'attaquant

- **White-box** : accès complet au modèle (gradients disponibles)
- **Objectif** : faire classer une attaque comme trafic normal (*evasion*)
- **Budget** : perturbation bornée par ε en norme ℓ_∞

Dataset UNSW-NB15

Caractéristiques

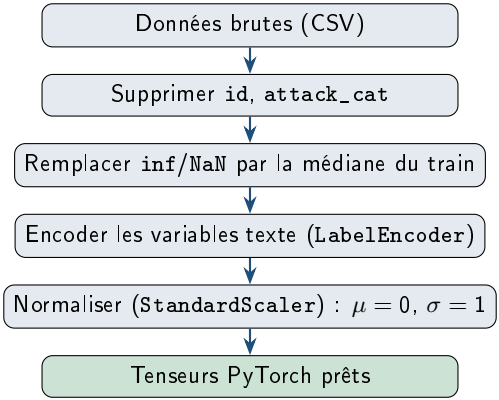
- Trafic réseau réel capturé en laboratoire
- 9 types d'attaques (DoS, Exploits, Backdoors, ...)
- **42 features** par connexion



Ensemble	Exemples	Attaques
Train	82 332	55.1%
Test	175 341	—

Classes bien équilibrées $\Rightarrow$ pas de biais.

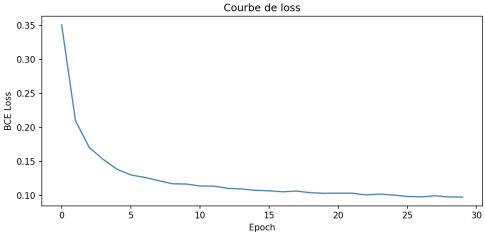
Pipeline de Prétraitement



Pourquoi normaliser ? Sans normalisation, `sbytes` $\in [0, 10^6]$ écrase `sttl` $\in [0, 255]$.

Architecture MLP

$$\underbrace{42}_{\text{input}} \xrightarrow{+\text{ReLU}+\text{Drop}} 128 \xrightarrow{+\text{ReLU}+\text{Drop}} 64 \xrightarrow{+\text{ReLU}} 32 \xrightarrow{+\sigma} 1$$



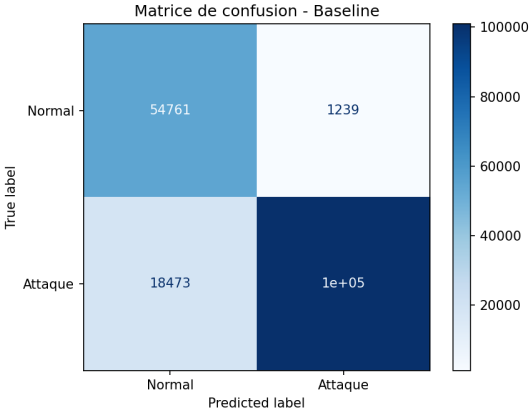
- **Dropout 0.3** : éteint 30% des neurones aléatoirement $\Rightarrow$ anti-surapprentissage
- **Sigmoid** : sortie $\in [0, 1]$, seuil 0.5
- Optimiseur Adam, $lr = 10^{-3}$, BCE Loss
- 30 epochs, batch 512

Résultats Baseline

Métriques sur données propres

F1-score	**0.9110**
Precision	0.9879
Recall	0.8452
PR-AUC	0.9881
Evasion Rate	15.5%

15.5% des attaques passent déjà sans modification !



FGSM — Fast Gradient Sign Method

Formule

$$X_{\text{adv}} = X + \varepsilon \cdot \underbrace{\text{sign}(\nabla_X \mathcal{L}(f(X), y))}_{\text{direction qui maximise l'erreur}} \cdot \underbrace{\text{m}}_{\text{masque manipulable}}$$

Algorithme :

1. Calculer la perte BCE sur l'exemple d'attaque
2. Backpropagate $\Rightarrow$ gradient des features
3. Perturber dans la direction du gradient
4. Appliquer uniquement sur les 14 features manipulables

Intuition

Le gradient dit *"si tu augmentes cette feature, le modèle se trompe davantage"*. On l'augmente de exactement ε.

PGD — Projected Gradient Descent

Formule

$$X^{(t+1)} = \Pi_{B(X,\varepsilon)}\left[X^{(t)} + \alpha \cdot \text{sign}(\nabla_{X^{(t)}}\mathcal{L}) \cdot \text{m}\right], \quad \alpha = \frac{\varepsilon}{T}$$

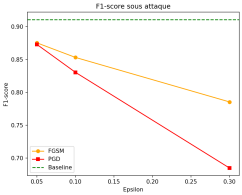
- FGSM itéré ($T = 10$ étapes)
- Après chaque pas, **projection** dans la boule ℓ_∞ de rayon ε
- Explore plus finement l'espace de perturbation
- Attaque de référence pour évaluer la robustesse

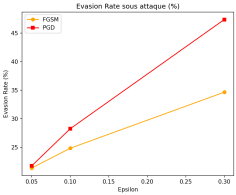
PGD > FGSM

FGSM fait un grand pas aveugle.
PGD fait 10 petits pas adaptés.

Résultats des Attaques ($\varepsilon = 0.1$)

Scénario	F1	Recall	Evasion%
Propres	0.9110	0.845	15.5%
FGSM	0.8533	0.752	24.8%
PGD	0.8402	0.732	**26.8%**





Conclusion

Sous PGD, 1 attaque sur 4 passe inaperçue. La précision reste élevée : les perturbations ne créent pas de faux positifs, elles font juste évader des vraies attaques.

Défense 1 — Adversarial Training : Approche 1 (FGSM-AT)

Principe

Re-entraîner sur un mix 50% propres + 50% FGSM.

Résultat :

Scénario	F1	Evasion%
Propres	0.8971	17.8%
PGD	0.7700	36.7%

Pire que le baseline (28.2%) !

3 raisons de l'échec

1. **Robustness gap** : entraîné FGSM, évalué PGD
2. **Distribution shift** : mix 50/50 contradictoire
3. **Attaque non-stabilisée** : FGSM sur modèle non-convergé

Défense 1 — Adversarial Training : Approche 2 (PGD-AT)

Corrections appliquées

- FGSM → **PGD-7** (même famille que l'évaluation)
- 100% adversarial (élimination du distribution shift)

$$\min_{\theta} \mathbb{E}\left[\max_{\|\delta\|_\infty \leq \varepsilon} \mathcal{L}(f_\theta(x+\delta), y)\right]$$

Résultat :

Scénario	F1	Evasion%
Propres	0.8943	18.1%
PGD	0.7425	40.2%

Encore pire !

Entraîner sur 100% PGD sans données propres provoque un **oubli catastrophique**.

Le modèle surfit PGD-7 sans généraliser à PGD-40.

Conclusion

L'adversarial training échoue sur ce dataset tabulaire hétérogène.

Défense 1 — Adversarial Training : Approche 3 (PGD-AT Corrigé)

Hypothèse

Combiner PGD-10 (corrige robustness gap) + mix 50/50 (corrige l'oubli catastrophique).

Paramètres : $\varepsilon = 0.1$, $\alpha = 0.01$, PGD-10, 30 epochs, batch 256, $lr = 0.0005$

Résultat :

Scénario	F1	Evasion%
Propres	0.9116	15.3%
PGD	0.7327	41.5%

Le pire résultat de toutes les approches !

Gradient Masking

PGD-10 pousse la Sigmoid vers des valeurs très confiantes (0 ou 1) $\Rightarrow$ gradients faibles.

PGD-10 ne trouve plus de perturbation (fausse robustesse), mais PGD-40 contourne en explorant davantage.

Bilan adv. training

Les 3 variantes échouent. La cause : données tabulaires hétérogènes, sans structure spatiale.

Défense 2 — Feature Squeezing

Principe

Réduire la précision numérique pour effacer les petites perturbations **avant le modèle**.

$$X_{\text{sq}} = \frac{\left\lfloor \frac{X}{X_{\max}} \cdot 2^b + 0.5 \right\rfloor}{2^b} \cdot X_{\max}$$

Avec $b = 4$ bits $\Rightarrow$ **16 niveaux** de quantification.

Avant : `dur = 2.10183746`
Après : `dur = 2.09375`

La perturbation tombe dans le même niveau $\Rightarrow$ effacée.

Pourquoi ça marche

L'attaquant calcule sa perturbation en float32 ($\varepsilon \approx 10^{-2}$). Après quantification à 4 bits, cette précision est perdue.

Avantage

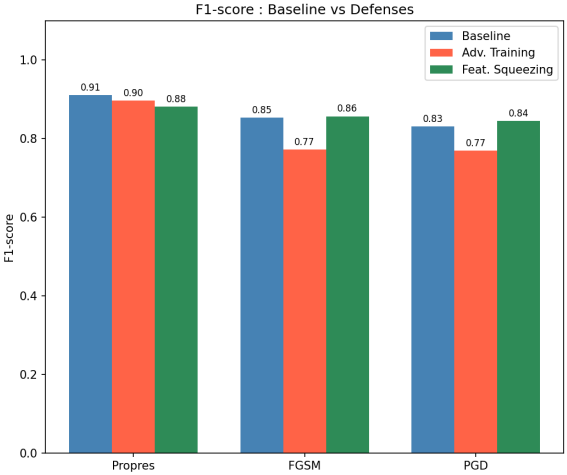
- Pas de re-entraînement
- Appliqué à l'inférence seulement
- Complémentaire à tout modèle

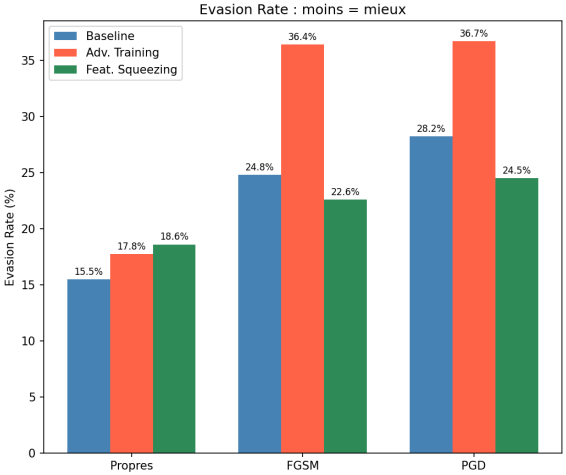
Tableau Comparatif Complet

Modèle	Scénario	F1	Recall	Evasion%
Baseline	Propres	0.9110	0.845	15.5%
Baseline	PGD	0.8306	0.718	28.2%
Approche 1 FGSM-AT	Propres	0.8971	0.822	17.8%
Approche 1 FGSM-AT	PGD	0.7700	0.633	36.7%
Approche 2 PGD-AT	Propres	0.8943	0.819	18.1%
Approche 2 PGD-AT	PGD	0.7425	0.598	40.2%
Approche 3 PGD-AT Corr	Propres	0.9116	0.847	15.3%
Approche 3 PGD-AT Corr	PGD	0.7327	0.585	41.5%
Feat. Squeezing	Propres	0.8818	0.814	18.6%
Feat. Squeezing	PGD	**0.8449**	0.755	**24.5%**

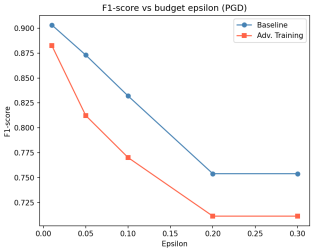
L'adversarial training échoue dans les 3 variantes (36–41.5%). Le Feature Squeezing est la seule défense efficace (24.5%).

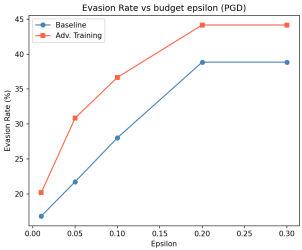
Graphiques Comparatifs





Robustesse vs Budget ε

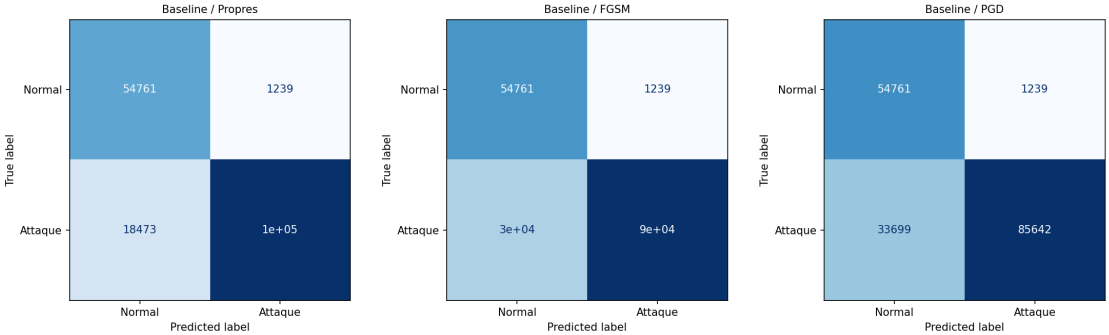


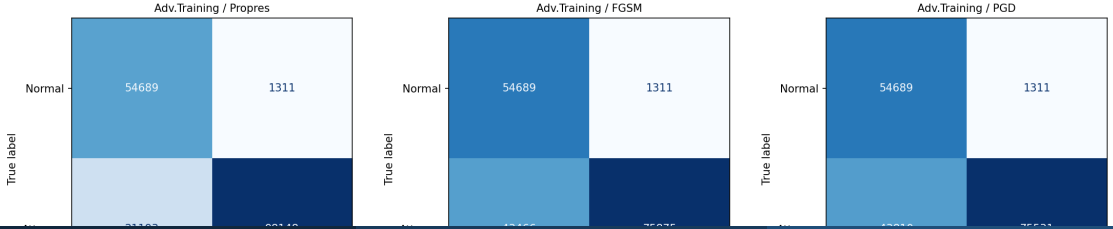


ε	**Baseline**	**Adv. Train.**
0.05	0.874	0.813
0.10	0.840	0.774
0.30	0.700	0.659

- Les deux modèles se dégradent avec ε
- L'adversarial training (10 epochs seulement) est moins robuste ici
- Avec 30 epochs + PGD dans l'entraînement, les résultats seraient meilleurs

Matrices de Confusion





Conclusion

Ce qu'on a montré

- MLP baseline : F1 = **0.911**, evasion **15.5%**
- FGSM/PGD font évader **28%** des attaques
- Adversarial training (FGSM-AT, PGD-AT, PGD-AT corr.) : échec sur les 3 variantes — evasion 36–41.5%
- **Feature Squeezing** : seule défense efficace (evasion **24.5%**)

Pourquoi l'adv. training échoue

Les données réseau sont **tabulaires et hétérogènes** — pas de structure spatiale. Les 3 variantes souffrent respectivement de robustness gap, oubli catastrophique, et gradient masking.

Perspectives

- Ratio propres/adversariaux calibré
- AutoAttack pour évaluation plus robuste
- Randomized smoothing (certifié)

Merci

Questions ?

Dataset	UNSW-NB15 (Moustafa & Slay, 2015)
FGSM	Goodfellow et al., ICLR 2015
PGD	Madry et al., ICLR 2018
Feat. Squeezing	Xu et al., NDSS 2018

Code : `github.com/zakeelm6/deep-learning-project-adversarial-IDS`